Informed Search

CSCI3230 (ESTR3108) | Tutorial 10

Wenxin Fan

Email: wxfan25@cse.cuhk.edu.hk

Office: Pentecostal Mission Hall Complex High Block, IMIXR Lab

Department of Computer Science and Engineering
The Chinese University of Hong Kong

November 2026



Learning Goals

1. Explain what a **Search Heuristic** is and compute $h(n)$ on a simple map.
2. Trace **Greedy Best-First Search** using a Priority Queue.
3. Trace **A* Search** using $g(n)$, $h(n)$, and $f(n)$.
4. Understand the key operations in the pseudocode and reconstruct a Solution Path.

Today's Focus: Definitions → Heuristics → Worked Examples → Pseudocode

From Uninformed Search to Informed Search

Uninformed Search
Uses information available in the Search Problem: Initial State, Successors, Step Cost, and Goal Test.

Informed Search
Uses additional **problem-specific knowledge** to guide which Search Node should be expanded.

Uniform-Cost Search
Chooses the node with the lowest **Path Cost** $g(n)$.

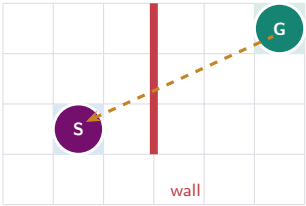
Search Heuristic
Estimates the remaining cost from a node to a Goal State.

Both Approaches Manage a Fringe; the Selection Rule Changes.

Why Do We Need a Search Heuristic?

Search Problem
Start at **S**. The Goal is **G**. From S, we can move **Up** or **Right**.

- Without information about the Goal, both moves may appear equally promising.
- Knowing the Goal's location helps us **estimate** which direction is preferable.



An Informed Strategy Uses an Estimate, Not a Guaranteed Future Path.

Search Heuristics: Definition

Heuristic Function $h(n)$
$h(n)$ = Estimated Cost of the Cheapest Path from node n to a Goal State.

Known Cost $g(n)$

From the Initial State to n

Estimated Remaining Cost $h(n)$

From n to the Goal State

If n is the Goal State, then $h(n) = 0$.

Heuristics Are Problem-Specific: Paths May Use Manhattan or Euclidean Distance.

Manhattan Distance and Euclidean Distance

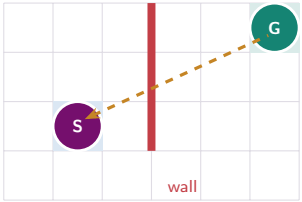
Manhattan Distance
$h_M(n) = |x_n - x_G| + |y_n - y_G|$

Euclidean Distance
$h_E(n) = \sqrt{(x_n - x_G)^2 + (y_n - y_G)^2}$

Example: $S = (1, 1)$, $G = (5, 3)$

Manhattan: $h_M(S) = 4 + 2 = \mathbf{6}$

Euclidean: $h_E(S) = \sqrt{20} \approx \mathbf{4.47}$



The red wall forces a detour; both estimates ignore it.

A Heuristic Estimates Remaining Cost Without Solving the Full Path.

Evaluation Function: Which Node Looks Best?

Evaluation Function $f(n)$
Assigns a priority to each Search Node. A **smaller** value is selected first from the Fringe.

Greedy Best-First Search

$$f(n) = h(n)$$

Only the estimated remaining cost.

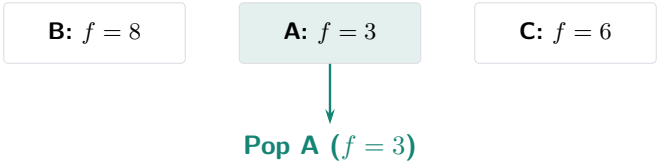
A* Search

$$f(n) = g(n) + h(n)$$

Known Path Cost + estimated remaining cost.

Both Algorithms Use a Priority Queue; Their Evaluation Functions Differ.

Priority Queue: Smallest Value First



Push
Insert a newly discovered Search Node with its priority.

PopMin
Remove the Search Node with the lowest current $f(n)$.

Priority Queue Order Is Determined by Evaluation Values, Not Arrival Time.

Greedy Best-First Search: Definition

Definition
Greedy Best-First Search expands the node that **appears closest to the Goal**, according to a Search Heuristic.

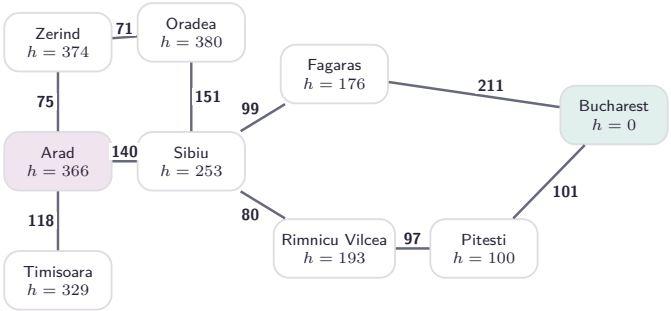
Evaluation Function $f(n) = h(n)$

Fringe
Priority Queue ordered by the lowest $h(n)$.

The known Path Cost $g(n)$ does **not** determine Greedy's priority.

Greedy Selects the Lowest Estimated Remaining Cost.

Greedy Worked Example: Romania Road Network

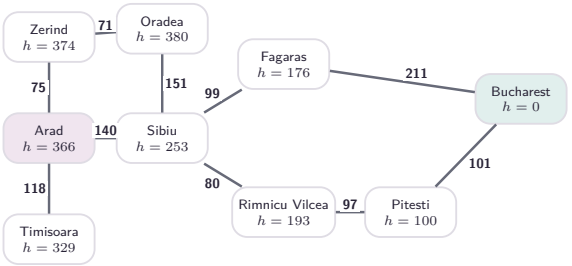


Start: **Arad** Goal: **Bucharest**

Road costs on edges; $h(n)$ estimates distance to Bucharest.

Which Neighbor Will Greedy Choose? Sibiu (253), Timisoara (329), or Zerind (374)?

Greedy Trace: Expand Arad



Pop Arad

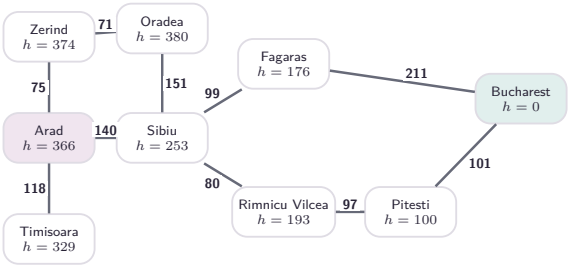
Generate **three** Successors.

Fringe Node	h
Sibiu	**253**
Timisoara	329
Zerind	374

Next Pop: Sibiu

Greedy Selects the Smallest $h(n)$ Across the Entire Fringe.

Greedy Trace: Expand Sibiu



Pop Sibiu

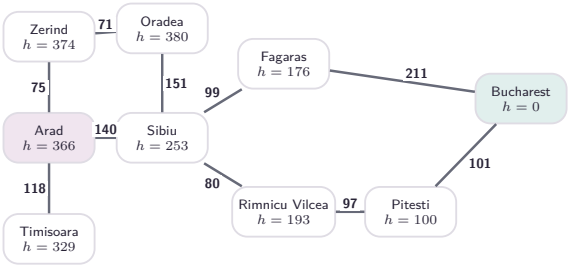
Add Fagaras, Rimnicu Vilcea, and Oradea.

Fringe Node	h
Fagaras	**176**
Rimnicu Vilcea	193
Timisoara	329
Zerind	374
Oradea	380

Next Pop: Fagaras

Fagaras Is Preferred Even Though Other Candidates Remain in the Fringe.

Greedy Trace: Expand Fagaras



Pop Fagaras

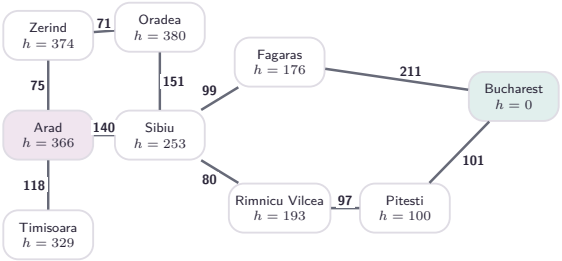
Generate Bucharest with $h = 0$.

Fringe Node	h
Bucharest	**0**
Rimnicu Vilcea	193
Timisoara	329
Zerind	374
Oradea	380

Next Pop: Bucharest

Goal Test Succeeds When Bucharest Is Popped from the Fringe.

Greedy Trace: Return the Path



Arad $\rightarrow$ Sibiu $\rightarrow$ Fagaras $\rightarrow$ Bucharest

$$\text{Path Cost} = 140 + 99 + 211 = \mathbf{450}$$

Greedy Used $h(n)$ to Choose Nodes; It Did Not Minimize $g(n)$.

Greedy Best-First Search: Pseudocode

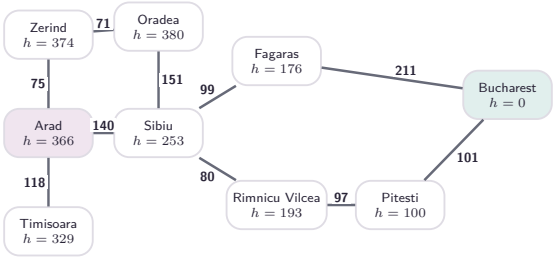
```
function Greedy(start):
  Fringe ← PriorityQueue(h(start), start)
  Discovered ← {start}
  Parent[start] ← None
  while Fringe not empty:
    u ← PopMin(Fringe)
    if GoalTest(u): return Path(u)
    for v in Successors(u):
      if v not in Discovered:
        Discovered.add(v)
        Parent[v] ← u
        Push(Fringe, (h(v),v))
  return Failure
```

Selection
PopMin: select the lowest $h(n)$.

Goal Test
Check whether the selected Search Node is the Goal.

Path
Use Parent links to reconstruct the Solution Path.

Why Can Greedy Take an Expensive Route?



Route	Selection Rule	Path Cost
Arad – Sibiu – Fagaras – Bucharest	Greedy: lowest h	450
Arad – Sibiu – Rimnicu Vilcea – Pitesti – Bucharest	Cheaper alternative	418

A Smaller $h(n)$ Is Only an Estimate of What Lies Ahead.

A* Search: Definition

Definition
A* Search combines the **known Path Cost** and the **estimated remaining cost** to evaluate Search Nodes.

A* Evaluation Function $f(n) = g(n) + h(n)$

$g(n)$
Exact Path Cost from the Initial State to node n.

$h(n)$
Estimated Cost from node n to the Goal State.

A* Pops the Fringe Node with the Smallest $f(n)$.

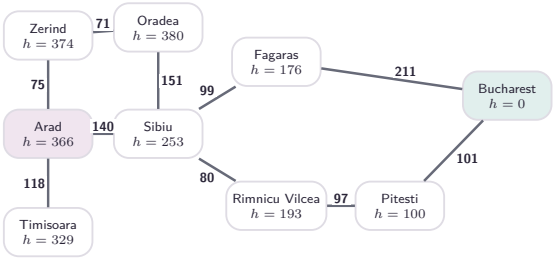
Compare the Three Evaluation Rules

Search Strategy	Priority	What It Considers
Uniform-Cost Search	$g(n)$	Actual Path Cost so far
Greedy Best-First Search	$h(n)$	Estimated remaining cost only
A* Search	$g(n) + h(n)$	Both known and estimated costs

$$\underbrace{g(n)}_{\text{start} \to n} + \underbrace{h(n)}_{n \to \text{goal}} = \underbrace{f(n)}_{\text{estimated total}}$$

The Priority Queue Is the Same Kind of Data Structure; Its Key Changes.

A* Worked Example: Read g, h, and f



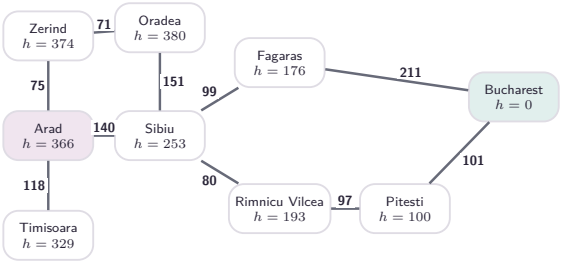
Initial State
Arad, with $g(\text{Arad}) = 0$.

Goal State
Bucharest, with $h(\text{Bucharest}) = 0$.

Search Rule
At every step, pop the node with the lowest $f(n) = g(n) + h(n)$.

Edge Labels: Actual Step Costs. **City Labels: Heuristic Estimates.**

A* Trace: Pop Arad



Pop Arad

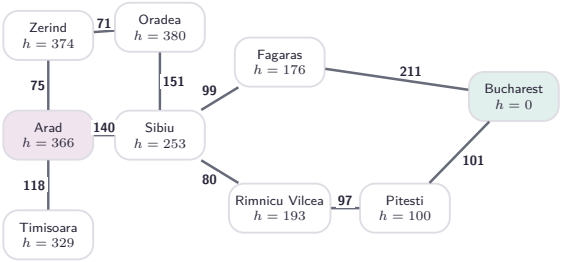
Generate three Successors; rank by $f = g + h$.

Node	g	h	f
Sibiu	140	253	**393**
Timisoara	118	329	447
Zerind	75	374	449

Next Pop: Sibiu: $f = 393$

A* Compares All Fringe Nodes by $f(n) = g(n) + h(n)$.

A* Trace: Pop Sibiu



Pop Sibiu

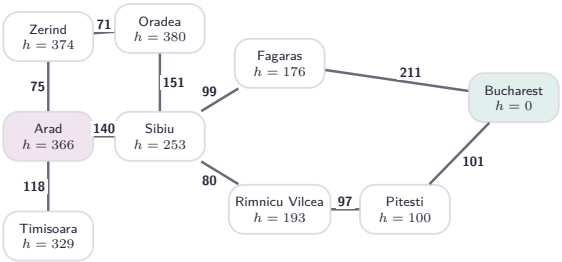
Generate Rimnicu, Fagaras, Oradea.

Node	g	h	f
Rimnicu	220	193	**413**
Fagaras	239	176	415
Timisoara	118	329	447
Zerind	75	374	449
Oradea	291	380	671

Next Pop: Rimnicu: $f = 413$

Rimnicu Vilcea Has the Lowest f, Even Though Fagaras Has Lower h.

A* Trace: Pop Rimnicu Vilcea



Pop Rimnicu

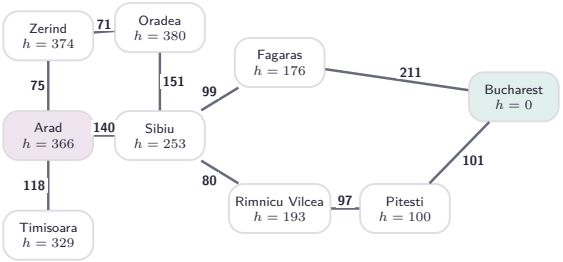
Generate Pitesti: $g = 220 + 97 = 317$.

Node	g	h	f
Fagaras	239	176	**415**
Pitesti	317	100	417
Timisoara	118	329	447
Zerind	75	374	449
Oradea	291	380	671

Next Pop: Fagaras: $f = 415$

A* Still Keeps the Alternative Routes on the Fringe.

A* Trace: Pop Fagaras



Pop Fagaras

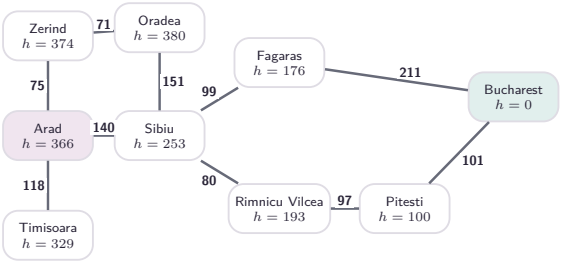
Generate Bucharest: $g = 239 + 211 = 450$.

Node	g	h	f
Pitesti	317	100	**417**
Timisoara	118	329	447
Zerind	75	374	449
Bucharest	450	0	450
Oradea	291	380	671

Next Pop: Pitesti: $f = 417$

Generating Bucharest Is Not Enough: Pitesti Has Smaller f.

A* Trace: Pop Pitesti and Relax Bucharest



Pop Pitesti

Via Pitesti, $g(\text{Bucharest}) = 317 + 101 = 418$.

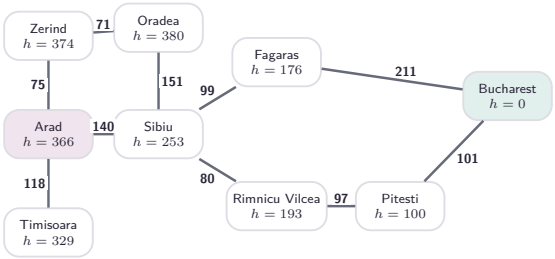
$$450 \rightarrow 418$$

Update **Best Cost** and **Parent** for Bucharest. **Next Pop: Bucharest:**

$f = 418$

A Cheaper Path Can Replace an Earlier Candidate for the Same State.

A* Trace: Pop the Goal and Return the Path



Arad $\rightarrow$ **Sibiu** $\rightarrow$ **Rimnicu Vilcea** $\rightarrow$ **Pitesti** $\rightarrow$ **Bucharest**

$$\text{Path Cost} = 140 + 80 + 97 + 101 = 418$$

A* Stops When the Lowest-f Search Node Popped from the Fringe Is the Goal.

A* Pseudocode I: Select and Test a Node

```
function AStar(start):
  Best[start] ← 0
  Parent[start] ← None
  Fringe ← PriorityQueue(
    (h(start), 0, start))
  while Fringe not empty:
    (f,g,u) ← PopMin(Fringe)
    if g ≠ Best[u]: continue
    if GoalTest(u): return Path(u)
    // Then expand the Successors
```

Priority
PopMin: choose the smallest $f(n) = g(n) + h(n)$.

Outdated Entry
If a better path was discovered later, skip the older Queue entry.

Goal Test
Check the Search Node **after popping** it.

A* Pseudocode II: Expand and Update Costs

```
for (v, stepCost) in Successors(u):
  newG ← g + stepCost
  if newG < Best.get(v,∞):
    Best[v] ← newG
    Parent[v] ← u
    newF ← newG + h(v)
    Push(Fringe,(newF,newG,v))
// End for; return Failure if empty
```

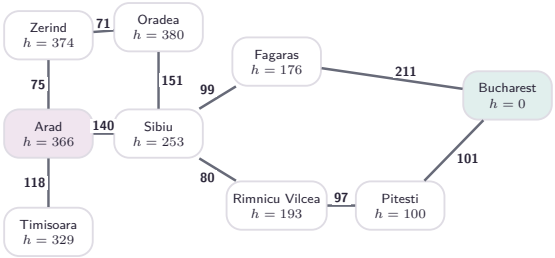
Relaxation
Compare a newly discovered g with the best known cost for the State.

Parent Update
Record the predecessor on the cheaper path.

Fringe Update
Push using the new $f = g + h$.

A* Reuses the Cost Update Mechanism from Uniform-Cost Search.

Why Not Stop When the Goal Is Generated?



First Generated Goal

Arad → Sibiu → Fagaras → Bucharest; Cost = **450**.

Improved Goal Path

Arad → Sibiu → Rimnicu Vilcea → Pitesti → Bucharest; Cost = **418**.

In A*, Generating a Goal Does Not Necessarily Mean It Has the Best Path Cost.

Admissible Heuristic: A Practical Check

Admissible Heuristic
A heuristic is **Admissible** if it never overestimates the true minimum remaining Path Cost $h^*(n)$.

$$0 \leq h(n) \leq h^*(n)$$

Safe Estimate
Straight-Line Distance cannot exceed the shortest drivable route in a normal road-distance problem.

Overestimate
A guessed value larger than the true remaining cost is **not** admissible.

Think Optimistic Estimate: $h(n)$ Never Claims a More Expensive Route Than Necessary.

Zero Heuristic: A* Reduces to UCS

Zero Heuristic
$h(n) = 0$ for every Search Node.

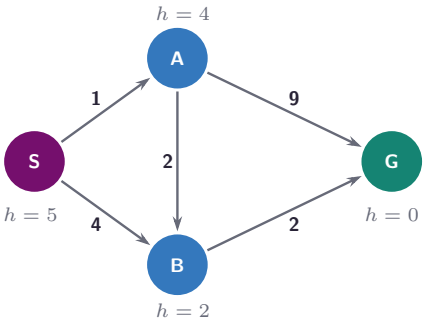
A* with Zero Heuristic
The priority depends only on known Path Cost. This is the same evaluation rule as **Uniform-Cost Search**.

$$f(n) = g(n) + 0 = g(n)$$

$$\underbrace{g(n) + h(n)}_{\text{A*}} \xrightarrow{h(n)=0} \underbrace{g(n)}_{\text{UCS}}$$

A Search Heuristic Provides the Extra Guidance Beyond UCS.

Practice: Greedy vs. A* on a Small Graph

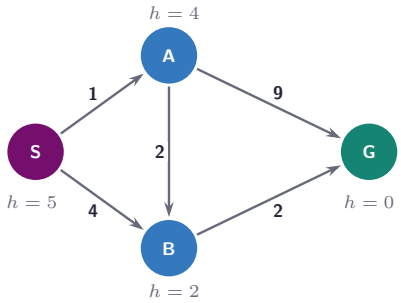


Rules
Start: **S**; Goal: **G**; Directed arcs; edge labels = Step Costs; h labels = estimated remaining costs.

1. Which Path does **Greedy** return?
2. Which Path does **A*** return?
3. What is each Path Cost?

Start with the Fringe. At Each Step, Calculate the Priority and PopMin.

Practice: First Expansion of S



Node	g	h	f
A	1	4	**5**
B	4	2	6

Greedy

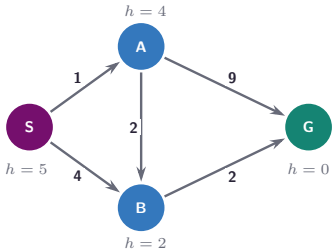
Compare h: pop **B** $(2 < 4)$.

A*

Compare f: pop **A** $(5 < 6)$.

The Algorithms Already Select Different Nodes After the First Expansion.

Practice Answers: Different Search Priorities



Strategy	Returned Path	Cost
Greedy	S $\rightarrow$ B $\rightarrow$ G	6
A*	S $\rightarrow$ A $\rightarrow$ B $\rightarrow$ G	**5**

Greedy prioritizes $h(n)$; A* prioritizes $g(n) + h(n)$.

Different Evaluation Functions Lead to Different Expansion Orders and Paths.

Three Questions to Remember

1. Heuristic Estimate

What does $h(n)$ estimate about the remaining Path Cost?

2. Which Node Next?

Greedy: smallest $h(n)$.

A*: smallest $g(n) + h(n)$.

3. How Do We Update?

Update the Best Cost and Parent; check the Goal on Pop.

Questions?

Search Heuristic → Evaluation Function → Fringe Selection → Returned Path.